

Develop AI Agents on Azure

Agents, tools, orchestration, and
interoperability

Rick Beerendonk
rick@oblicum.com

What You'll Learn

- Explain what turns a chat model into an agent
- Give an agent tools, and decide who is allowed to run them
- Connect an agent to company knowledge and to Microsoft 365
- Choose an orchestration shape when one agent is not enough
- Recognise when to use MCP and when to use A2A

Problem: A Chat App Cannot Do Anything

A chat model produces text. Ask it to "book the meeting room" and it produces a *description* of booking a meeting room.

- ✗ It cannot look up today's data
- ✗ It cannot call your systems
- ✗ It stops after one answer, even if the job needs five steps

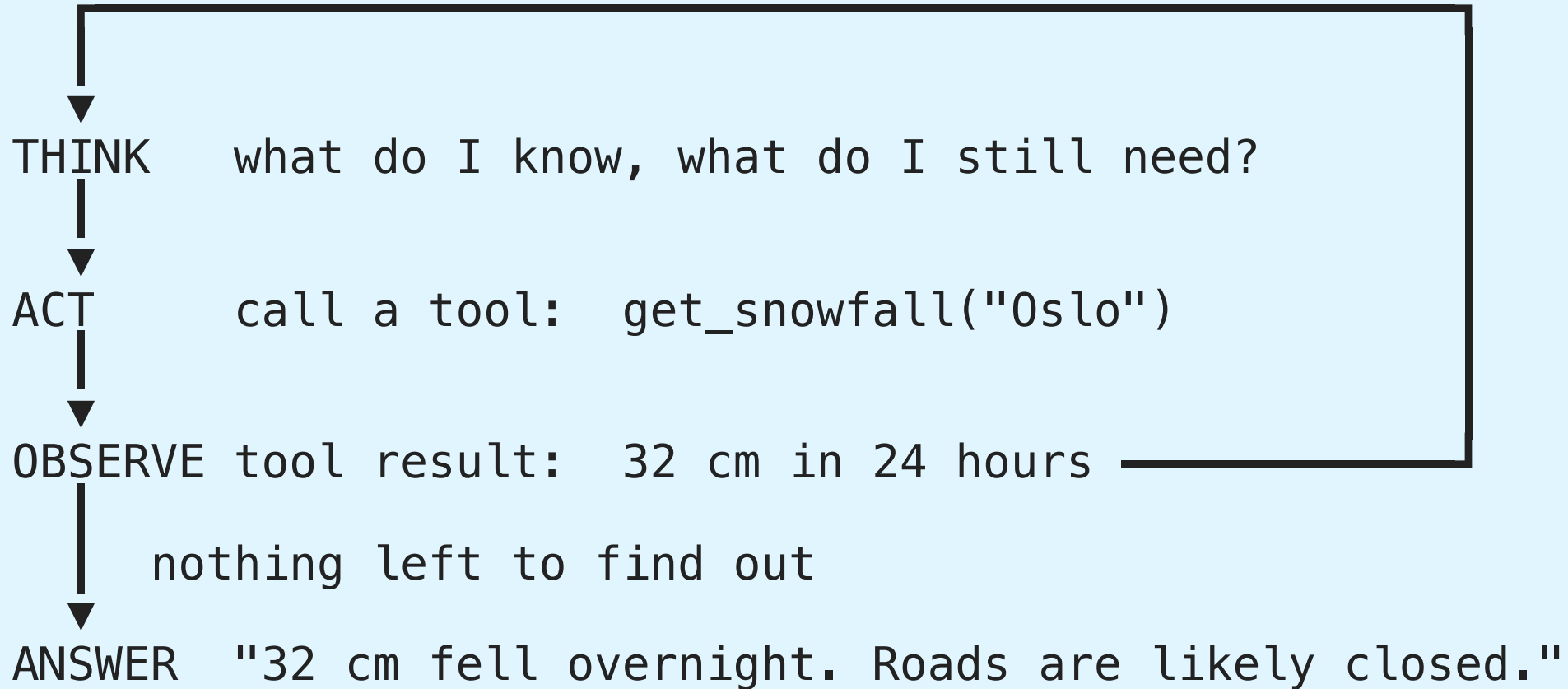
Need: something that can think, act, look at the result, and keep going until the job is done.

Agent = model + instructions + tools + a loop.

The loop is the part people forget. It is the whole trick.

The Agent Loop

user goal: "Is it snowing in Oslo, and should I cancel the trip?"



The model decides when to stop looping. Instructions, tool descriptions, and approval settings are how you keep that decision under control.

Retrieval Practice — Lab 1

Pause here and complete [Lab 1](#) before continuing.

Agent Service

What is an AI Agent?

[EXAM]

Problem: LLMs respond to a single prompt; they cannot take multi-step actions

Agent = model + instructions + tools + automatic tool calling

Agent types in Microsoft Foundry:

Type	Sub-type	Description
Declarative	Prompt-based	Model + instructions + tools + prompts (most common)
Declarative	Workflow	Multi-agent orchestration defined in YAML
Hosted	—	Containerized code-based agents hosted by Foundry platform

Agent Service Features

Feature	Description
Automatic tool calling	Agent decides when and how to invoke tools
Securely managed data	Conversation data managed via Responses API
Extensive tool catalog	Functions, code interpreter, file search, MCP servers
Enterprise security	Keyless auth, content safety integration
Customizable storage	Platform-managed or bring your own Azure Blob
Observability/tracing	Built-in trace logging
Model selection	Any deployed model

Required resources: Foundry project + model deployment

Optional: Azure AI Search, Azure Storage, Azure Key Vault, Azure Functions

Development Approaches

Approach	Best for
Foundry portal	No-code visual design, playground testing
VS Code extension (Microsoft Foundry)	Code-first, YAML, version control

VS Code Agent YAML pattern:

```
# yaml-language-server: $schema=https://aka.ms/ai-foundry-vsc/agent/1.0.0
version: 1.0.0
name: my-agent
model:
  id: "gpt-4.1"
  options:
    temperature: 0.5
    top_p: 1
instructions: |
  You are a helpful assistant.
tools: []
```

Create Agent via SDK

[EXAM]

```
from azure.ai.projects import AIProjectClient
from azure.ai.projects.models import PromptAgentDefinition

project_client = AIProjectClient(
    credential=DefaultAzureCredential(),
    endpoint="https://{resource-name}.services.ai.azure.com/api/projects/{project-name}"
)

agent = project_client.agents.create_version(
    name="my-agent",
    definition=PromptAgentDefinition(
        model="gpt-4.1",
        instructions="You are a helpful assistant.",
        tools=[],
    )
)

agent = project_client.agents.get(name="my-agent")    # retrieve an existing agent
```

Keyless access: `DefaultAzureCredential()` plus the **get** operation retrieves an existing agent through Entra ID. An API-key credential is not Entra ID, and **create** or **delete** do not return the existing agent.

Custom Tools

Custom Tool Options

[EXAM]

Option	Description
Custom function (function calling)	Describe structure; agent decides when to invoke
Azure Functions	Event-driven via queue triggers; input/output bindings
OpenAPI specification	Connect to any HTTP API via OpenAPI 3.0 spec
Azure Logic Apps	Low-code/no-code workflow integration

Function Calling

```
from azure.ai.projects.models import FunctionTool, PromptAgentDefinition

function_tool = FunctionTool(
    name="recent_snowfall",
    parameters={
        "type": "object",
        "properties": {
            "location": {"type": "string", "description": "City name"},
        },
        "required": ["location"],
        "additionalProperties": False
    },
    description="Get recent snowfall totals for a given location.",
    strict=True,
)

agent = project_client.agents.create_version(
    name="snowfall-agent",
    definition=PromptAgentDefinition(
        model="gpt-4.1",
        instructions="Help users find snowfall data.",
        tools=[function_tool],
    )
)
```

strict=True enforces the parameter schema

Azure Functions Tool

```
from azure.ai.projects.models import (
    AzureFunctionTool, AzureFunctionDefinition,
    AzureFunctionBinding, AzureFunctionStorageQueue
)

tool = AzureFunctionTool(
    azure_function=AzureFunctionDefinition(
        input_binding=AzureFunctionBinding(
            storage_queue=AzureFunctionStorageQueue(
                queue_name="INPUT_QUEUE",
                queue_service_endpoint="STORAGE_QUEUE_ENDPOINT",
            )
        ),
        output_binding=AzureFunctionBinding(
            storage_queue=AzureFunctionStorageQueue(
                queue_name="OUTPUT_QUEUE",
                queue_service_endpoint="STORAGE_QUEUE_ENDPOINT",
            )
        ),
        function=AzureFunctionDefinitionFunction(
            name="queue_trigger",
            description="Process a request",
            parameters={...},
        ),
    )
)
```

Communication via Storage Queue (input + output bindings)

OpenAPI Tool

Supports OpenAPI 3.0 only

```
from azure.ai.projects.models import OpenApiTool, OpenApiAnonymousAuthDetails

tool = OpenApiTool(
    openapi=OpenApiFunctionDefinition(
        name="get_weather",
        spec=openapi_weather,      # dict loaded from JSON/YAML file
        description="Get weather data",
        auth=OpenApiAnonymousAuthDetails(),
    )
)
```

Supported auth types: anonymous, API key, managed identity

OpenAPI Authentication and Connections

[EXAM]

When the API needs a key on every call, declare an **API-key security scheme** in the OpenAPI spec so the platform attaches it automatically:

```
from azure.ai.projects.models import OpenApiConnectionAuthDetails
auth = OpenApiConnectionAuthDetails(connection_id=project_connection.id)
```

Approach	Result
API-key security scheme	Key injected on every request — defined once
Project connection	Reuses the key already stored in the project connection
Adding the header manually	Repeats secret handling in code and risks leaking the key
Bearer/OAuth scheme	Expects an OAuth token, not an API key
Identity passthrough	Forwards the caller's identity , not the stored API key

Project Connections — One Credential, Many Agents

[EXAM]

Problem: every agent and every application configures the Azure AI Search key for itself.

Solution: add a **connection** to the Search resource in the Foundry project, then reference that connection from each agent and app.

Approach	Result
Project connection to Azure AI Search	One place to store, rotate, and audit the credential
A custom HTTP connection per app	Duplicated configuration and administration
One managed identity per app	Duplicated role assignments, still configured per app
Calling Search directly from each app	The search configuration is never shared

Not credential management: enabling RBAC and disabling key-based access **hardens** the resource; a managed private endpoint controls **network** access. Neither centralizes the credential.

Isolation still applies: give each agent its **own identity** for tool and data access, so permissions can be granted, audited, and revoked per agent.

MCP Tools

Why MCP Exists

Problem: every tool you hand-code lives inside one agent. Ten agents that need the same GitHub tools means ten copies to build, secure, and update.

Without MCP

agent A — own GitHub code
agent B — own GitHub code
agent C — own GitHub code

3 copies to maintain

With MCP

agent A }
agent B } → GitHub MCP server
agent C } (owns the tools)

1 server, updated in one place

MCP = Model Context Protocol. A standard way for a *server* to publish tools and for any agent to consume them.

Mental model: MCP is a power socket. The agent brings the plug; it does not need to know how the power station works.

MCPTool — Remote MCP Servers

[EXAM]

No manual session/client management needed when using built-in MCPTool:

```
from azure.ai.projects.models import MCPTool, PromptAgentDefinition

mcp_tool = MCPTool(
    server_label="GitHub",
    server_url="https://api.githubcopilot.com/mcp/",
    allowed_tools=["list_repos"],      # optional: restrict accessible tools
    require_approval="never"         # or "always"
)
mcp_tool.update_headers({"Authorization": "Bearer {token}"})

agent = project_client.agents.create_version(
    name="github-agent",
    definition=PromptAgentDefinition(
        model="gpt-4.1",
        instructions="Help with GitHub tasks.",
        tools=[mcp_tool],
    )
)
```

MCP Tool Approval

[EXAM]

require_approval="always" → agent response contains **mcp_approval_request**

```
# Check for approval request in the response
if response_item.type == "mcp_approval_request":
    # Present to user or auto-approve
    await respond_with_approval(
        approval_request_id=response_item.id,
        approve=True
    )
```

Key advantages of MCP vs. hard-coded tools:

- **"Integrate once"** — tools updated on server; no agent redeployment needed
- **LLM-agnostic** — switch underlying model without reworking integrations
- **Standardized auth** — consistent authentication mechanism across tools

Retrieval Practice — Lab 2

Pause here and complete [Lab 2](#), then check its solution.

Foundry IQ — Knowledge

Controlling Tool Use — `tool_choice`

[EXAM]

By default the model decides. When an agent sometimes answers **without** calling its knowledge tool, the citations are ungrounded — force the call.

Value	Behavior
<code>tool choice="auto"</code>	Default — the model may skip every tool

```
response = client.responses.create(  
    model=DEPLOYMENT,  
    input=messages,  
    tools=[mcp_tool],  
    tool_choice="required",  
)
```

Required vs specific: `"required"` guarantees a tool runs; it does not say *which*. To restrict one run to public web results only, name the tool: `{"type": "bing_grounding"}`. Naming `{"type": "azure_ai-search"}` would instead force the private index.

What is Foundry IQ?

[EXAM]

Microsoft's managed knowledge platform for AI agents, built on Azure AI Search

- Multiple agents share the same knowledge bases
- Uses MCP internally to connect agents to knowledge bases

RAG steps: Retrieve → Augment → Generate

Supported data sources:

Data Source	Access type	Best for
Azure AI Search Index	Indexed	Existing search investment
Azure Blob	Direct	Files (PDF, DOCX, TXT, MD,

Classic RAG vs Agentic RAG

[EXAM]

Classic RAG searches once with the user's words, then answers.

```
question ---> one search ---> generate
```

Agentic RAG gives retrieval to the model as a tool, so it plans the search itself.

```
question ---> rewrite / split into sub-questions
                | searches run in parallel
                v
            read results ---> not enough? search again ---> generate
```

Choose	When
Classic RAG	One index, FAQ-style questions, fixed cost and latency
Agentic RAG	Answer needs several chunks , earlier turns must steer retrieval, or sources must be searched in parallel

Not the same as: *iterative retrieval* (repeats a query without conversation-aware planning) or *chain of thought* (structures reasoning, not retrieval).

Connecting an Agent to a Knowledge Base

[EXAM]

```
from azure.ai.projects.models import MCPTool, PromptAgentDefinition

knowledge_tool = MCPTool(
    server_label="product-docs",
    server_url=f"{search_endpoint}/knowledgebases/product-documentation/mcp"
)

agent = project_client.agents.create_version(
    agent_name="product-support-agent",
    definition=PromptAgentDefinition(
        model="gpt-4o-mini",
        instructions="Answer product questions. Always cite your sources.",
        tools=[knowledge_tool]
    )
)
```

The Search credential comes from the **project connection**, not from application code.

Retrieval Instructions Pattern

Three critical behaviors to specify:

```
retrieval_instructions = """
CRITICAL RULES:
- You must ALWAYS search the knowledge base before answering any question
- You must NEVER answer from your own knowledge or training data
- Every answer must include citations in this format:
  【doc_id:search_id|source_name】
- If the knowledge base doesn't contain the answer, respond with:
  "I don't have that information in my knowledge base."
"""
```

Retrieval Practice — Lab 4

Pause here and complete [Lab 4](#), then check its solution.

Microsoft 365 Integration

Publishing an Agent to Teams/Copilot

[EXAM]

Publishing creates an Agent Application with:

- Dedicated invocation URL (stable across version updates)
- Agent identity (distinct Microsoft Entra app registration)
- Creates Azure Bot Service resource
- User data isolation

Publishing scopes:

Scope	Visibility	Requires admin approval
Shared	"Your agents" in Teams	No — immediate
Organization	"Built by your org" in Teams	Yes

Required roles: Azure AI Project Manager (publish), Azure AI User (invoke)

Microsoft.BotService provider must be registered in the subscription

Post-publish: Reassign RBAC to the published agent's new identity for Azure resources (AI Search, Storage) — dev-time permissions do not transfer

Microsoft 365 Publishing Options

Direct Foundry publishing:

- Creates an Agent Application with a stable invocation URL
- Creates an Azure Bot Service resource and Microsoft Entra application
- Generates a Microsoft 365 publishing package
- Routes new agent versions through the same public endpoint

Other channels:

Channel	Use case
Teams and Microsoft 365 Copilot	Users work inside Microsoft 365
Web application preview	Browser-based demonstrations and stakeholder testing
Stable API endpoint	Embed the agent in a custom application
Azure Bot Service channels	Slack, Telegram, Twilio, Facebook, and other

Use the Microsoft 365 Agents Toolkit when the solution needs custom single sign-on, middleware, or multi-environment deployment.

Access Microsoft 365 Data with Work IQ

Work IQ gives an agent access to authorized workplace context such as:

- Emails and conversations
- Meetings and calendar information
- Documents and collaboration data

Important boundaries:

- The signed-in user's Microsoft 365 permissions still apply
- Request only the data needed for the task
- Explain which data sources the agent can use
- Validate citations and permissions before exposing results

Work IQ is different from Foundry IQ:

	Foundry IQ	Work IQ
Primary data	Configured enterprise knowledge sources	Microsoft 365 workplace data
Typical use	Product, policy, and document knowledge	Personal and organizational work context

Test and Operate a Published Agent

Before publishing:

1. Test normal, ambiguous, and adversarial prompts in the Foundry playground
2. Verify every configured tool and knowledge source
3. Check citations, authentication, and user-data isolation
4. Confirm the published identity has the required Azure RBAC roles

After publishing:

- Test in Teams and Microsoft 365 Copilot with representative users
- Monitor invocation failures, latency, tool calls, and user feedback
- Update the agent version without changing the stable invocation URL
- Reassign permissions whenever a published identity is newly created

Common failure: the agent works in Foundry but fails after publishing because development-time permissions do not transfer to the published identity.

Agent Workflows

Workflow Patterns

[EXAM]

For a workflow that must pause before an external side effect, use an **ask_question** step and continue only when the returned value equals **"approved"**. This is a workflow-variable approval pattern, distinct from the SDK's typed **approve=True** tool-approval contract.

Pattern	Description
---------	-------------

Workflow Node Types:

Node	Sub-types / Purpose
------	---------------------

Choosing an Orchestration

[EXAM]

Requirement	Choose
Deterministic steps, conditions, and shared state with minimal development effort	A workflow
Ordered work with approval before a risky update	Sequential template + an Ask a question node
Agents decide who contributes next	Group chat
Independent steps that can run at the same time	Concurrent

Why not the alternatives: a multi-agent group chat is built for autonomous conversational collaboration and lets the agents choose the order; concurrent orchestration runs steps in parallel; threads and runs without a workflow — or separate agent runs coordinated in your own application code — mean you write and maintain the orchestration yourself.

Retrieval Practice — Lab 5

Pause here and complete [Lab 5](#), then check its solution.

Power Fx Formulas in Workflows

[EXAM]

Logic in workflow nodes:

Variables: `Local.*` (captured during execution), System variables (workflow context)

Interpolation in message text: wrap the expression in braces — `{Upper(Local.Var01)}`. Without braces the formula is printed as literal text.

Power Fx — variable scopes & operators

Scope prefix is required — bare `Var1` does not resolve.

Prefix	Source
<code>Local.<name></code>	Workflow variables you defined
<code>System.<name></code>	Built-in (e.g. <code>System.RunId</code>)
<code>Trigger.<name></code>	Fields from the triggering message / input
<code>Activity.<NodeName>.Output.<field></code>	Output of an upstream node

Boolean operators: `&&` `||` `Not(...)` `=` `<>` `>` `<` `>=` `<=`

If/Else node — "must have a value"

[EXAM]

The **If/Else node** takes a **boolean expression** (not a value-returning `If(...)`).

Goal	Condition
Variable must have a value	<code>Not(IsBlank(Local.Var1))</code>
Variable is missing	<code>IsBlank(Local.Var1)</code>
List has at least one item	<code>Not(IsEmpty(Local.ItemList))</code>
Two values both required	<code>Not(IsBlank(Local.A)) && Not(IsBlank(Local.B))</code>

IsBlank vs IsEmpty:

- `IsBlank(x)` — true when `x` is `null` or an empty string `""`
- `IsEmpty(coll)` — true when a **collection / table** has zero rows

The function `If(cond, a, b)` **returns a value** — use it inside *Set Variable*, not as the condition of an If/Else node.

Invoke a Workflow from Code

```
conversation = openai_client.conversations.create()

stream = openai_client.responses.create(
    conversation=conversation.id,
    extra_body={"agent": {"name": "triage-workflow", "type": "agent_reference"}},
    input="Users can't reset their password from the mobile app.",
    stream=True,
)

for event in stream:
    if event.type == "response.completed":
        print(event.response.output_text)
```

Check action status: `event.item.type ==`
`ItemType.WORKFLOW_ACTION` → `event.item.status`

Microsoft Agent Framework

Framework Overview

Open-source SDK; provider-agnostic (Azure OpenAI, OpenAI, Anthropic, Copilot Studio, Foundry)

vs. Foundry Agent Service:

	Microsoft Agent Framework	Foundry Agent Service
Compute	Your infrastructure	Fully managed
Storage	Your storage	Platform-managed
Providers	Any (OpenAI, Anthropic, ...)	Foundry models
Control	Full code control	Configuration-based

Core Components

Component	Description
Agent	Main class: client + instructions + tools
AzureOpenAIResponsesClient	Connection + auth to Foundry project
AgentSession	Persistent conversation history (roles: USER, ASSISTANT, SYSTEM, TOOL)
Built-in tools	Code Interpreter, File Search, Web Search
@tool decorator	Marks Python functions as agent tools

Custom Tools with @tool

```
from microsoft_agent_framework import Agent, tool
from pydantic import Field
from typing import Annotated

@tool(name="get_stock_price", description="Get current price for a stock ticker")
def get_stock_price(
    ticker: Annotated[str, Field(description="Stock ticker symbol, e.g. MSFT")]
) -> float:
    return 428.50

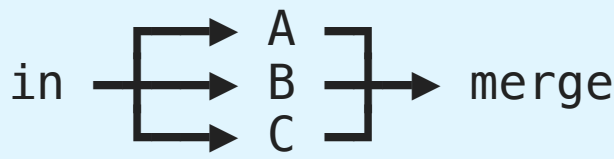
agent = Agent(
    client=AzureOpenAIResponsesClient(...),
    instructions="You are a financial advisor.",
    tools=[get_stock_price]
)
session = AgentSession()
result = await agent.run("What is the price of MSFT?", session=session)
```

Use **Annotated** + **Field** from Pydantic for parameter descriptions

Multi-Agent Orchestration

Orchestration Patterns — the Shapes

Each pattern is really a picture of *who talks to whom, in what order*.

Sequential	$A \rightarrow B \rightarrow C$	each one refines the last
Concurrent		all at once, then combine
Group chat	$A \leftrightarrow B \leftrightarrow C$	a manager picks who speaks next
Handoff	$A \text{ --?--> } B \text{ or } C$	A decides who should take over
Magentic	$\text{plan} \rightarrow A \rightarrow \text{replan} \rightarrow C$	plan first, adapt as it goes

Ask two questions to pick one: Do the steps depend on each other? Is the order known before you start?

Orchestration Patterns

[EXAM]

Pattern	Builder class	When to use	Avoid when
Sequential	<code>SequentialBuilder</code>	Step-by-step pipelines; progressive refinement	Stages can run in parallel
Concurrent	<code>ConcurrentBuilder</code>	Parallel, independent tasks; ensemble; voting	Tasks depend on each other
Group chat	<code>GroupChatBuilder</code>	Collaborative conversation; maker-checker; human-in-the-loop	Simple linear tasks; speed critical
Handoff	<code>WorkflowBuilder</code> + switch-case	Dynamic delegation; expertise unknown upfront	Fixed agent order known
Magentic	<code>MagenticBuilder</code>	Complex open-ended; plan-first; dynamic task ledger	Simple/deterministic tasks

All patterns: `participants()` → `build()` → `run()` or `run_stream()`
Results: `get_outputs()` → iterate messages with `.author` and `.content`

Workflow Edge Types

Edge	Description
Direct	Sequential: $A \rightarrow B$
Conditional	Execute only when condition is met
Switch-Case	Route to different executors based on predefined cases
Fan-Out	Send one message to multiple executors simultaneously
Fan-In	Combine outputs from multiple executors into one

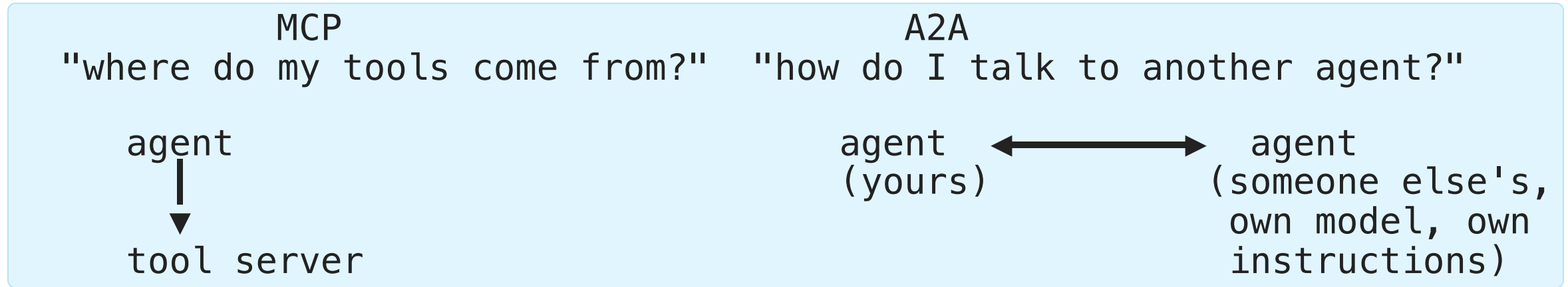
Built-in events: WorkflowStartedEvent, WorkflowOutputEvent, WorkflowErrorEvent, ExecutorInvokeEvent, ExecutorCompleteEvent, RequestInfoEvent

Workflows support **checkpointing** (save/resume state)

A2A Protocol

MCP and A2A Are Not Competitors

They solve two different questions.



Question	MCP	A2A
Other side is...	a tool provider	another autonomous agent
Who picks the model?	you, for your agent	each agent picks its own
Discovery	server URL + tool list	agent-card.json

One sentence: MCP gives an agent *hands*; A2A gives it *colleagues*.

Agent-to-Agent (A2A)

Standardized framework for agent discovery, communication, and coordinated task execution

Key advantage over MCP: Each A2A agent chooses its own LLM; integrated authentication; cross-vendor

Agent Card — discovery document at `/.well-known/agent-card.json`:

```
{
  "name": "weather-agent",
  "description": "Provides weather forecasts",
  "endpoint_url": "https://my-agent.example.com/",
  "capabilities": { "streaming": true },
  "skills": [
    {
      "id": "weather-forecast",
      "name": "Weather Forecast",
      "description": "Get weather for a location",
      "tags": ["weather", "forecast"],
      "examples": ["What is the weather in Paris?"]
    }
  ]
}
```

A2A Server and Client

Server components:

Component	Role
Agent Card	Describes capabilities; at <code>/well-known/agent-card.json</code>
Request Handler	Routes requests to <code>AgentExecutor</code> ; manages Task Store
Server Application	Starlette + Uvicorn (ASGI); listens on HTTP

AgentExecutor interface:

- `execute` — processes requests; sends results via `EventQueue`
- `cancel` — handles task cancellation

Client request types:

- **Non-streaming** — send message, wait for complete response
- **Streaming** — incremental responses for long-running tasks

Advanced Generative AI and Agent Operations

Operate agents by separating runtime state, tools, evaluation, and monitoring.

Agent Lifecycle and Runtime State

[EXAM]

Agent: instructions, model, and tools

Thread or conversation: persistent context

Message: user, assistant, or tool content

Run: one execution against the conversation

Use a durable conversation or persisted message store when history must survive sessions.

To keep a full case history across sessions, store the conversation ID and reuse it. Saving only the final model response loses the tool calls and tool outputs; summarizing condenses history instead of restoring the complete execution record.

Agent Memory and Scopes

[EXAM]

Memory is not RAG and not conversation history.

Feature	Stores
Conversation	The full turn history of one ongoing case
Knowledge	Source documents retrieved as evidence

Scopes decide who shares a memory:

Scope	Lifetime
session	Discarded when the session ends
"{{\$userId}}"	Per signed-in user. across sessions — no ID in code

```
memory_tool = MemoryTool(scope="{{$userId}}")

agent = project_client.agents.create_version(
    name="support-agent",
    definition=PromptAgentDefinition(model="gpt-4.1", tools=[memory_tool]),
)
```

The `{{$userId}}` placeholder is substituted at runtime, so each user is isolated without generating identifiers yourself.

Evaluate, Control, and Monitor Agents

[EXAM]

- Use a groundedness gate or evaluator to stop unsupported answers
- Use **Search Index Data Reader** for read-only Search access
- Use reflection to regenerate incomplete answers
- Treat **approve=True** as a typed approval decision, not a string
- Monitor tool calls, latency, tokens, handoffs, quality, safety, and versions

Error analysis: inspect retrieval, prompt, model, tool, orchestration, safety, and user-experience layers in order.

Evaluate an Agent in a Pipeline

[EXAM]

Which evaluators for a RAG agent?

Evaluator	Answers
Retrieval	Did the search return the right documents?
Groundedness	Does the answer follow those documents?
Fluency and coherence	Readability and logical flow — not correctness

Gate a pull request with GitHub Actions:

- Authenticate with **OIDC** — federated, no long-lived stored credential
- Set the **project endpoint** so the workflow knows which Foundry project to evaluate against
- **Fail the workflow** when evaluation fails, so an unsafe version cannot merge

The evaluation config path, the model deployment name, and the tenant ID do not identify the project.

Key Takeaways

1. **Agent types:** Declarative (prompt-based / workflow) + Hosted
2. **Custom tools:** Function calling, Azure Functions, OpenAPI 3.0, Logic Apps
3. **MCP:** `MCPTool` attaches remote MCP servers; `require_approval` controls autonomy
4. **Foundry IQ:** Managed RAG platform; multiple data sources; citation pattern
5. **Agent Framework:** Open-source, provider-agnostic; `@tool` decorator for function tools
6. **Orchestration patterns:** Sequential, Concurrent, Group chat, Handoff, Magentic
7. **A2A:** `/.well-known/agent-card.json` for discovery; Agent Skills describe capabilities

Problem: "The Agent Gave a Bad Answer"

That sentence is not a bug report. It could mean any of these:

- The search returned the wrong document
- The instructions never said to cite sources
- The tool threw an error and the model guessed instead
- The model is too small for the task
- Everything worked — last week's version was simply different

Need: vocabulary precise enough to say *which part* failed.

This presentation is that vocabulary.

Retrieval Practice — Lab 6

Pause here and complete [Lab 6](#), then check its solution.

Classic Agent Service Lifecycle

[EXAM]

For the exam, distinguish the persistent conversation objects from one execution:

Agent	= instructions + model + tools
Thread	= persistent conversation state
Message	= user, assistant, or tool content in a thread
Run	= one execution of an agent against a thread

Typical flow:

```
create agent
  → create thread
  → add user message
  → create and process run
  → inspect run status
  → list messages
```

Why it matters: a thread stores context, while a run represents the model and tool execution that produces the next response.

Current API note: newer Responses API and Agent Framework examples may use

Thread vs Run — the Whiteboard and the Meeting

THREAD (the whiteboard – stays on the wall)

msg 1	user	"Compare Q1 and Q2 revenue"
msg 2	assistant	"Q1 was 4.2M, Q2 was 5.1M"
msg 3	user	"Now chart it"



RUN #1



RUN #2

(a meeting: reads the whole board, may call tools,
writes the next message, then ends)

The thread persists; a run is over as soon as it produces its message. Cost, latency, tool calls, and failures all belong to a *run*. Context belongs to the *thread*.

Retrieval Practice — Lab 7

Pause here and complete [Lab 7](#), then check its solution.

Tool Taxonomy

[EXAM]

Tool	Primary capability	Typical control
Function tool	Application-owned operation	Your schema, validation, and execution
MCP tool	Discoverable remote tools	Server permissions and approval policy
File search	Retrieval from uploaded files or vector stores	Vector store and citation controls

Selection rule: use the narrowest tool that solves the task, and require approval for actions with external side effects.

Azure AI Search RBAC

[EXAM]

For read-only agent queries against a shared Azure AI Search service, assign the agent identity the **Search Index Data Reader** role. Managed identities let each agent receive a separate, auditable data-plane permission.

Role boundaries:

- **Search Index Data Reader:** query documents in indexes
- **Search Index Data Contributor:** read and push documents
- **Search Service Contributor:** manage indexes, indexers, and data sources

Use the narrowest data-plane role that satisfies the agent's task; do not grant service-management permissions to a retrieval-only agent.

Retrieval Practice — Lab 8

Pause here and complete [Lab 8](#) before continuing.

Generation Controls

[EXAM]

Parameter	Main effect	Exam consideration
temperature	Randomness and variation	Lower for repeatable structured output
top_p	Probability-mass vocabulary cutoff	Usually tune instead of temperature, not both
max_tokens / max_output_tokens	Output budget	Affects completeness, cost, and rate-limit reservation
frequency_penalty	Reduces repeated tokens	Useful when responses repeat phrases
presence_penalty	Encourages new topics	Useful when broader exploration is desired

Do not use a parameter to solve the wrong problem: grounding fixes missing knowledge; a penalty does not fix hallucination, and temperature does not add facts.

Retrieval Practice — Lab 9

Pause here and complete [Lab 9](#), then check its solution.

Reflection and Self-Critique Loops

[EXAM]

A quality loop asks a second evaluator or agent to inspect the first answer.

draft answer

- check against requirements and source context
- identify unsupported claims or missing steps
- revise or reject
- return answer with evidence

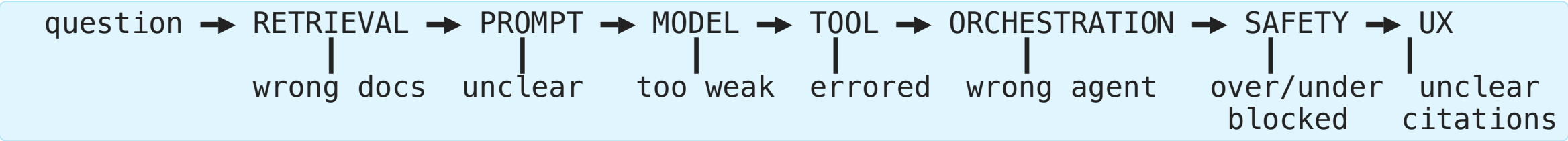
Useful checks:

- Is every claim grounded in retrieved context?
- Does the answer satisfy the requested format?
- Did the tool call use valid and authorized arguments?
- Should a human review this result?

Evaluation and Error Analysis

[EXAM]

A wrong answer has a *first* broken link. Walk the chain in order and stop at the first failure.



Separate failures by layer:

Failure	Diagnostic question
Retrieval	Did the index return the right source and chunk?
	Were instructions, context, and examples

Use a labeled evaluation dataset, reproduce the failure, change one layer, and compare the before/after metrics.

Production Agent Monitoring

[EXAM]

Monitor more than HTTP success:

- Run duration and time spent in each model or tool call
- Prompt, completion, and tool tokens
- Rate-limit responses and retries
- Tool-call validity, failures, and approval requests
- Groundedness, relevance, coherence, fluency, and safety scores
- Handoff routes, loop counts, and abandoned runs
- Model, prompt, agent, and knowledge-source versions

Alert on: quality regression, rising latency, cost spikes, repeated tool errors, unsafe outputs, and retrieval failures.

MSLearn status: tracing, evaluation, and user feedback are covered; this complete operational view is primarily an exam synthesis.

Key Takeaways

1. **Thread:** persistent context; **run:** one execution; **message:** content within the thread
2. **Tools:** choose the narrowest safe tool and control external side effects
3. **Parameters:** temperature, top P, token limits, frequency penalty, and presence penalty solve different problems
4. **Reflection:** draft, inspect, revise, and measure the added cost
5. **Error analysis:** isolate retrieval, prompt, model, tool, orchestration, safety, and UX failures
6. **Operations:** monitor tokens, latency, tools, quality, safety, versions, and retries
7. **MSLearn relationship:** the building blocks are present, but this exam-oriented lifecycle and operations synthesis is not a

Human Approval Contract

[EXAM]

When a tool call requires approval, the SDK submits a Boolean decision such as `approve=True` or `approve=False` tied to the tool call. The string `"approved"` is not the SDK approval value.

Retrieval Practice — Lab 3

Pause here and complete [Lab 3](#) before continuing.

Retrieval Practice — Lab 10

Pause here and complete [Lab 10](#), then check its solution.

Appendix

Relationship to the Microsoft Learn path

Coverage Map

Exam objective	MSLearn status
Agent thread / run / message lifecycle	Indirect — Agent Framework sessions and workflow runs are covered; the classic lifecycle is not central
Function, MCP, file search, code interpreter, and Bing grounding tools	Partial — most tools are covered; the exam grouping is not presented as one comparison
Prompt parameters and generation controls	Partial — temperature, top P, and token limits are covered; frequency and presence penalties are not
Reflection, self-critique, and error analysis	Indirect — evaluation is covered; these patterns are not developed deeply
Agent production monitoring	Indirect — tracing and feedback appear, but the exam checklist is broader